```
# Create and train the model
X, y = make_regression(n_samples=100, n_features=1, noise=0.1) model = LinearRegression()
model.fit(X, y)

# Save the model
joblib.dump(model, 'linear_regression_model.pkl') # Load the model
loaded_model = joblib.load('linear_regression_model.pkl') print("Loaded model prediction:",
loaded_model.predict([[2.0]])) This approach ensures your scikit-learn models are easily saved and
reloaded for future use.
```

Saving Models with TensorFlow

TensorFlow simplifies the process of saving and exporting deep learning models. It supports saving either the entire model or just the architecture and weights separately. Below is an example: import tensorflow as tf

```
from tensorflow.keras.models import Sequential from tensorflow.keras.layers import Dense # Define
and train the model
model = Sequential([
    Dense(10, activation='relu'), Dense(1)
])
model.compile(optimizer='adam', loss='mse') model.fit([[1], [2], [3]], [[2], [4], [6]], epochs=100,
verbose=0) # Save the model
model.save('my_model')

# Load the model
loaded_model = tf.keras.models.load_model('my_model') print("Loaded model prediction:",
loaded_model.predict([[4]])) TensorFlow’s .save and .load_model methods ensure comprehensive model
preservation, including architecture, optimizer configuration, and weights.
```

Saving Models with PyTorch

PyTorch offers flexibility in saving models by allowing developers to save the entire model or just the state dictionary (parameters). Here’s how: import torch

```
import torch.nn as nn

# Define a simple model
class SimpleModel(nn.Module):
    def __init__(self):
        super(SimpleModel, self).__init__() self.fc = nn.Linear(1, 1)

    def forward(self, x):
        return self.fc(x)

# Initialize and train the model
model = SimpleModel()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01) criterion = nn.MSELoss()
x = torch.tensor([[1.0], [2.0], [3.0]]) y = torch.tensor([[2.0], [4.0], [6.0]]) for epoch in range(100):
```